KALASETU

Art Marketplace Platform



PROJECT REPORT

Submitted By:

Name: Shrikant Aabasaheb Sargar

Class: SYCS

Roll Number: SYCS37

Subject: Android Application Development

Under the Guidance of: Ms. Nancy Nelson

Bhavan’s College Mumbai 400059,

Maharashtra2026

INDEX

SR. NO.	TOPICS	PAGE NO.	SIGNATURE	REMARK
1	**Preliminary Investigation:** • Aim of the project • Problem Statement • Description of the Application • Number of Modules • Advantages of the developed application • Technologies Used	___	_____	_____
2	**System Analysis:** • Event Table	___	_____	_____
3	**UML Diagrams:** • ERD (Entity Relationship Diagram) • Use Case Diagram • Class Diagram • Object Diagram • Activity Diagram • Sequence Diagram	___	_____	_____
4	**System Coding:** • List of Modules • List of Tables with Attributes • Program Description with Code • Test Cases, Test Data and Test Results • Screen Layouts	___	_____	_____
5	Future Enhancements	___	_____	_____
6	Conclusion	___	_____	_____
7	References and Bibliography	___	_____	_____

1. PRELIMINARY INVESTIGATION

1.1 AIM OF THE PROJECT

The aim of the **Kalasetu** project is to design and develop a **secure, reliable, and centralized digital art marketplace** that acts as a bridge between **artists** and **art buyers**, ensuring authenticity, quality control, and transparency in the process of online art selling.

In the traditional art ecosystem, many talented artists face difficulties in reaching potential buyers due to limited exposure, lack of digital platforms, and absence of structured approval mechanisms. At the same time, users who wish to purchase artwork often struggle to verify the authenticity and quality of paintings. Kalasetu aims to overcome these challenges by introducing a **role-based system** where every artwork undergoes an **admin approval process** before being displayed for sale.

1.2 PROBLEM STATEMENT

In the current scenario, the process of selling and purchasing artwork is largely **unorganized, manual, and limited in reach**. Many artists depend on physical exhibitions, local galleries, or social media platforms to sell their paintings. These methods have several limitations, such as restricted audience reach, lack of centralized management, and absence of quality assurance.

Additionally, there is no structured system that:

- Validates artwork before public display
- Allows artists to track approval or rejection status
- Ensures that only approved content reaches users
- Maintains a secure record of purchases and orders

The **Kalasetu** project addresses these problems by introducing an **admin approval workflow**, role-based access control, and a structured database-driven system that ensures authenticity, reliability, and smooth interaction between artists and buyers.

1.3 DESCRIPTION OF THE APPLICATION

- **Kalasetu** is a role-based digital art marketplace application designed to connect artists and users through a secure and admin-controlled system.
- The application allows artists to upload their paintings along with required details such as title, price, description, and image.
- Every uploaded painting is first sent to the administrator for verification to ensure authenticity and quality.
- The administrator has the authority to approve or reject paintings based on predefined guidelines.
- Only paintings approved by the admin are made visible to users for browsing and purchase.
- Users can view paintings, explore artwork details, and buy paintings through the application.
- The application maintains transparency by allowing artists to view the approval status of their paintings.
- The system securely stores user data, painting data, and order details in the database.
- Kalasetu provides a centralized platform that eliminates manual processes and increases the reach of artists.
- The application ensures trust, security, and smooth interaction among artists, administrators, and users.

1.4 Number Of Module

1 User Module

- This module manages all functionalities related to users (buyers) of the application.
- Users can register and log in using secure authentication methods implemented through the users table.
- User details such as name, username, phone number, address, password, and secret code are stored securely in the database.
- The module supports login validation using username and password.
- Users can recover their account credentials using phone number and secret code.
- Users can view approved paintings uploaded by artists.
- Users can place orders for paintings they wish to purchase.
- Users can view order history and order status after purchasing paintings.
- All user-related operations are handled using database queries defined in the DBHelper class.

2 Artist Module

- This module handles all operations related to artists registered in the system.
- Artists can register and log in securely using credentials stored in the artists table.
- Artists can upload paintings by providing painting title, description, price, and image.
- Uploaded paintings are stored in the paintings table with default status set to PENDING.
- Artists can view all their uploaded paintings along with their approval status.
- The module allows artists to track whether a painting is approved or rejected by the admin.
- Artists can view incoming orders placed by users for their paintings.

- Artists can view detailed order information, including user details and delivery address.
- Artists can update order status (such as accepted or completed).

3 Admin Module

- This module provides administrative control over the entire application.
- The admin can view all registered users and artists in the system.
- The admin can view all paintings uploaded by artists, especially paintings with PENDING status.
- The admin has the authority to approve or reject paintings uploaded by artists.
- Only paintings approved by the admin are made visible to users.
- The admin can manage artists and users, including viewing and deleting records if required.
- The admin ensures quality control and authenticity of artwork displayed on the platform.
- This module plays a critical role in maintaining trust and transparency in the system.

4 Order Management Module

- This module manages the purchase and order lifecycle of paintings.
- When a user buys a painting, an order record is created in the orders table.
- Each order stores information such as painting ID, user username, artist username, order status, and timestamp.
- Artists can view orders received for their paintings.
- Users can track the status of their orders.
- The module supports updating order status to reflect processing or completion.
- Order details are linked with painting, user, and artist information using database relationships.
- This module ensures smooth coordination between users and artists after a purchase is made.

1.5 ADVANTAGES OF THE DEVELOPED APPLICATION

1. Centralized Art Marketplace

- The application provides a single, centralized platform where artists can upload paintings and users can browse and purchase artwork without relying on multiple sources.

2. Admin-Controlled Approval System

- Every painting uploaded by an artist is verified by the administrator before being displayed, ensuring quality, authenticity, and trustworthiness.

3. Improved Trust for Users

- Users can confidently purchase paintings knowing that all displayed artwork has been approved and verified by the admin.

4. Increased Visibility for Artists

- Artists gain wider exposure for their artwork beyond physical galleries, helping them reach more potential buyers.

5. Transparent Workflow for Artists

- Artists can view the approval status (Pending, Approved, or Rejected) of their uploaded paintings, maintaining transparency in the system.

6. Secure User and Artist Authentication

- Login and registration functionalities ensure that only authorized users, artists, and admins can access the system.

7. Efficient Order Management

- The system allows smooth handling of orders, including order placement, tracking, and status updates for both users and artists.

8. Easy Accessibility

- Users can access the platform anytime and from anywhere, increasing convenience for both buyers and artists..

9. Promotion of Art and Culture

- The platform supports and promotes creative artwork by providing artists with a digital space to showcase their talent.

1.6 TECHNOLOGIES USED

- .

1. Android Studio

- Android Studio is used as the Integrated Development Environment (IDE) for designing, developing, debugging, and testing the application.

2. Kotlin Programming Language

- Kotlin is used as the primary programming language for application development due to its concise syntax, null safety, and full support in Android Studio.

3. SQLite Database

- SQLite is used as the local database to store user, artist, painting, and order details in a structured manner.

4. SQLiteOpenHelper

- The SQLiteOpenHelper class is used to manage database creation, versioning, and data operations such as insert, update, delete, and query.

5. XML (Extensible Markup Language)

- XML is used for designing the user interface layouts, defining buttons, text fields, and views.

6. Cursor and ContentValues

- Cursor is used to retrieve data from the database, while ContentValues is used to insert and update data efficiently.

7. Android SDK

- Android Software Development Kit (SDK) provides essential libraries, APIs, and tools required to build and run the Android application.

8. Draw.io / UML Tools

- Diagram tools such as Draw.io are used to design ER diagrams, use case diagrams, sequence diagrams, and activity diagrams.

9. Emulator / Android Device

- Android Emulator and real Android devices are used for testing and validating the functionality of the application.

2. SYSTEM ANALYSIS

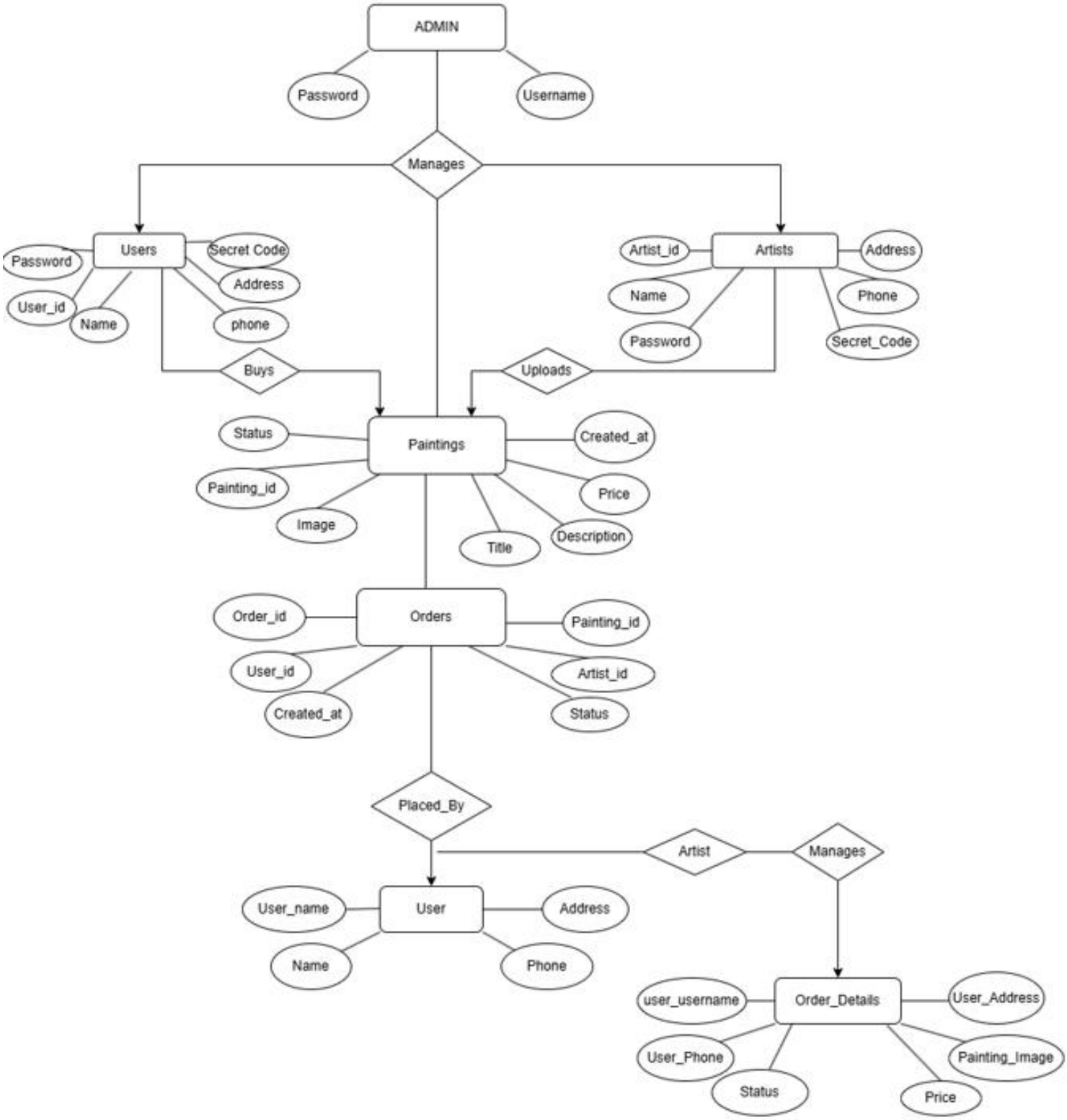
EVENT TABLE

Event No.	Event	Actor	Trigger	System Response	Database Table
E1	User Registration	User	Submit signup form	Validate and store user data	users
E2	User Login	User	Enter login credentials	Authenticate user	users
E3	Artist Registration	Artist	Submit signup form	Validate and store artist data	artists
E4	Artist Login	Artist	Enter login credentials	Authenticate artist	artists
E5	Upload Painting	Artist	Upload painting details	Store painting as PENDING	paintings
E6	View Pending Paintings	Admin	Open admin dashboard	Retrieve pending paintings	paintings
E7	Approve Painting	Admin	Approve painting	Update painting status	paintings
E8	Reject Painting	Admin	Reject painting	Update painting status	paintings
E9	View Approved Paintings	User	Open gallery	Display approved paintings	paintings
E10	Place Order	User	Click Buy button	Create new order	orders
E11	View Orders	Artist	Check orders	Display received orders	orders

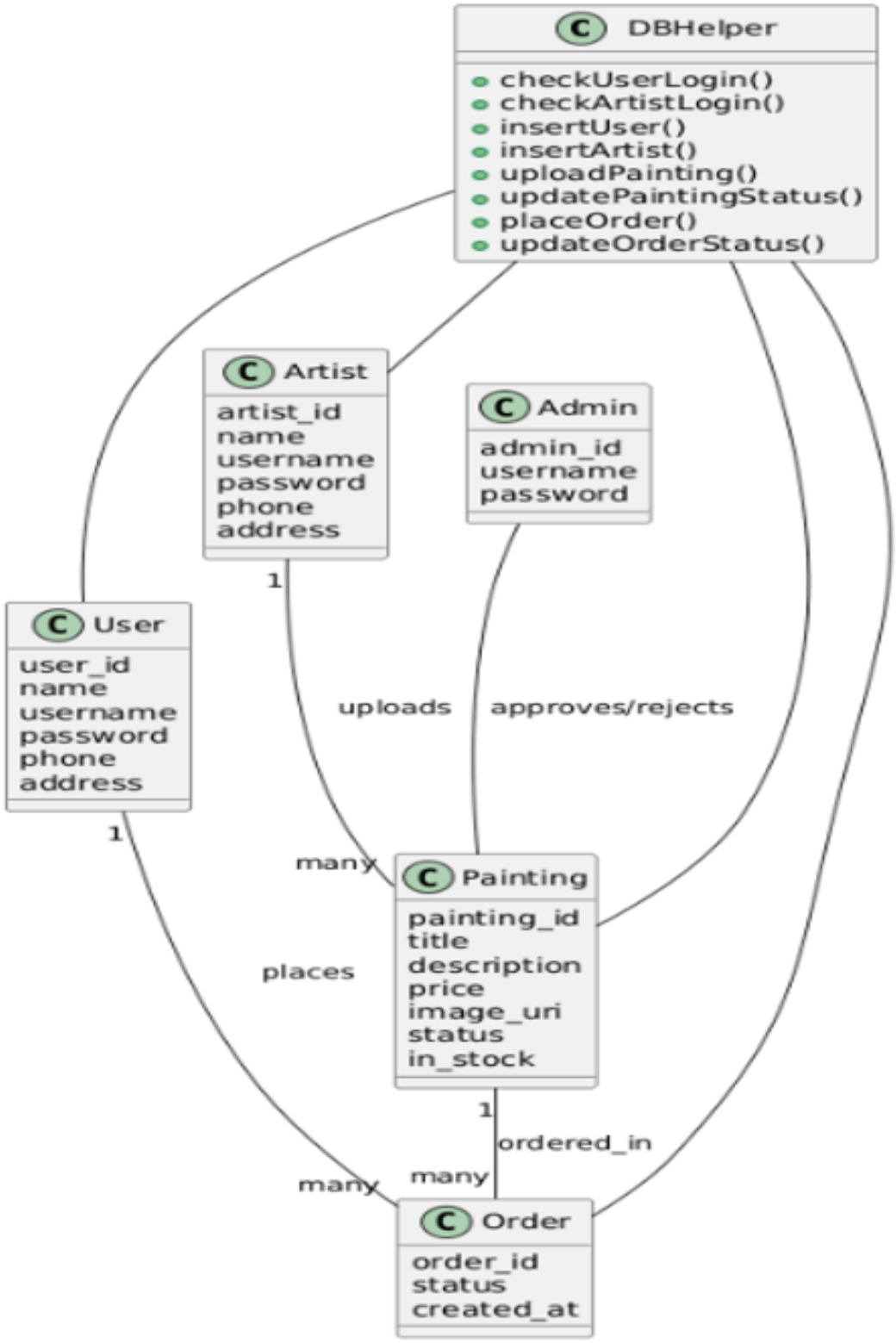
E12	Update Order Status	Artist	Change order status	Update order record	orders
E13	View Order History	User	Open order history	Display past orders	orders
E14	Password Recovery	User/Artist	Forgot password	Validate secret code	users / artists
E15	Delete User/Artist	Admin	Delete record	Remove data	users / artists

3.UML DIAGRAMS

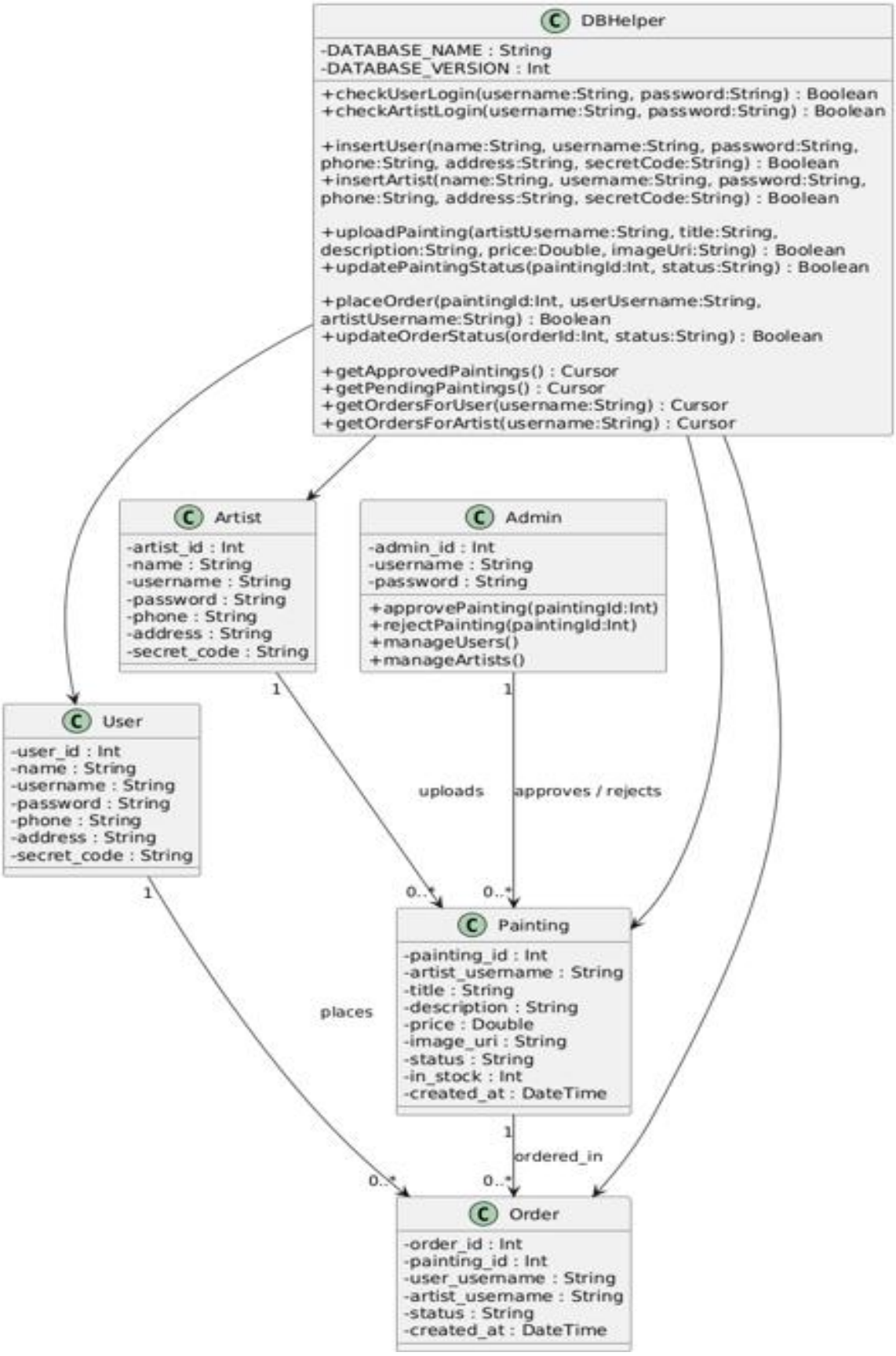
1.ER DIAGRAM (ENTITY RELATIONSHIP DIAGRAM)



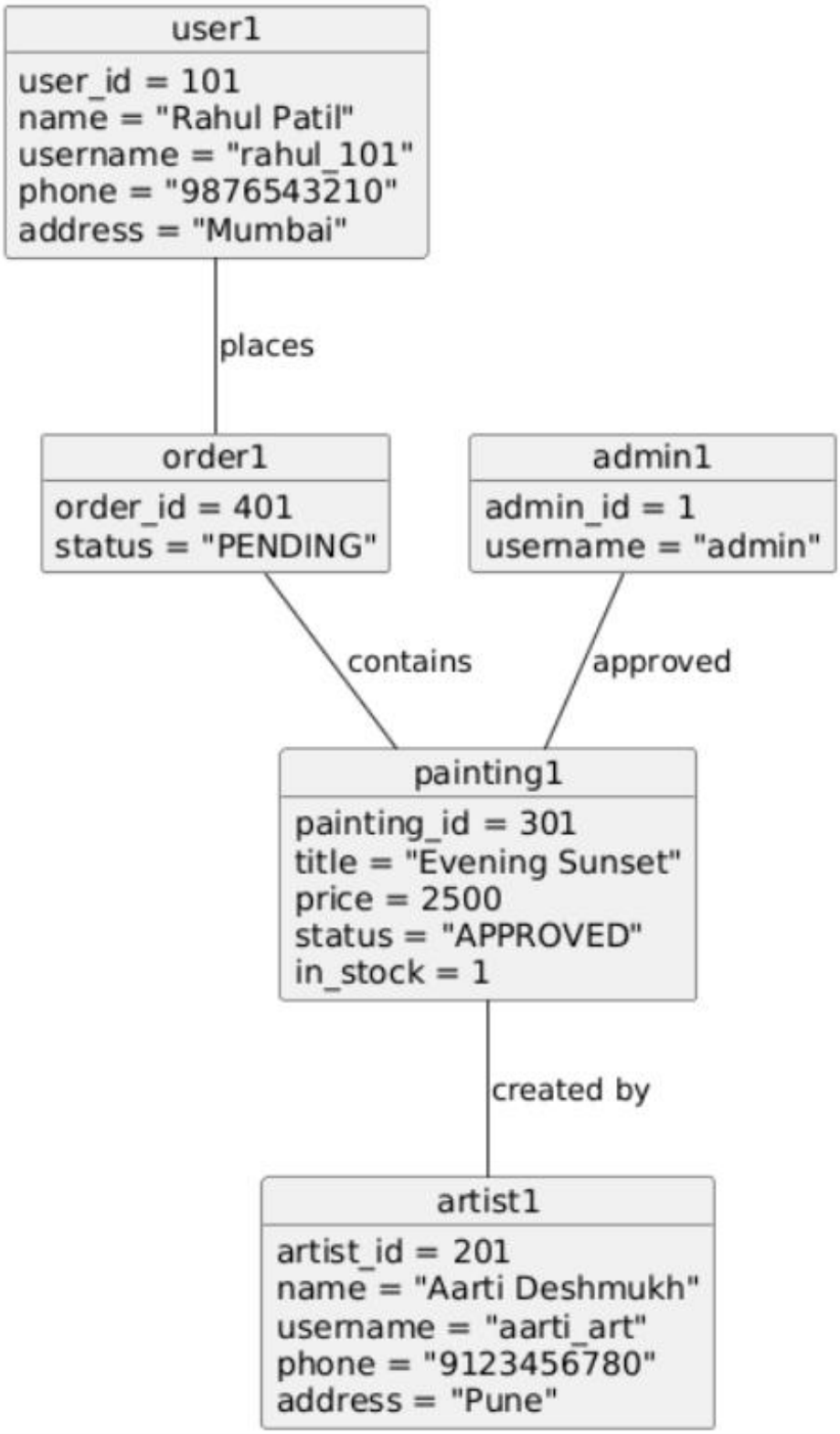
2. Use Case Diagram



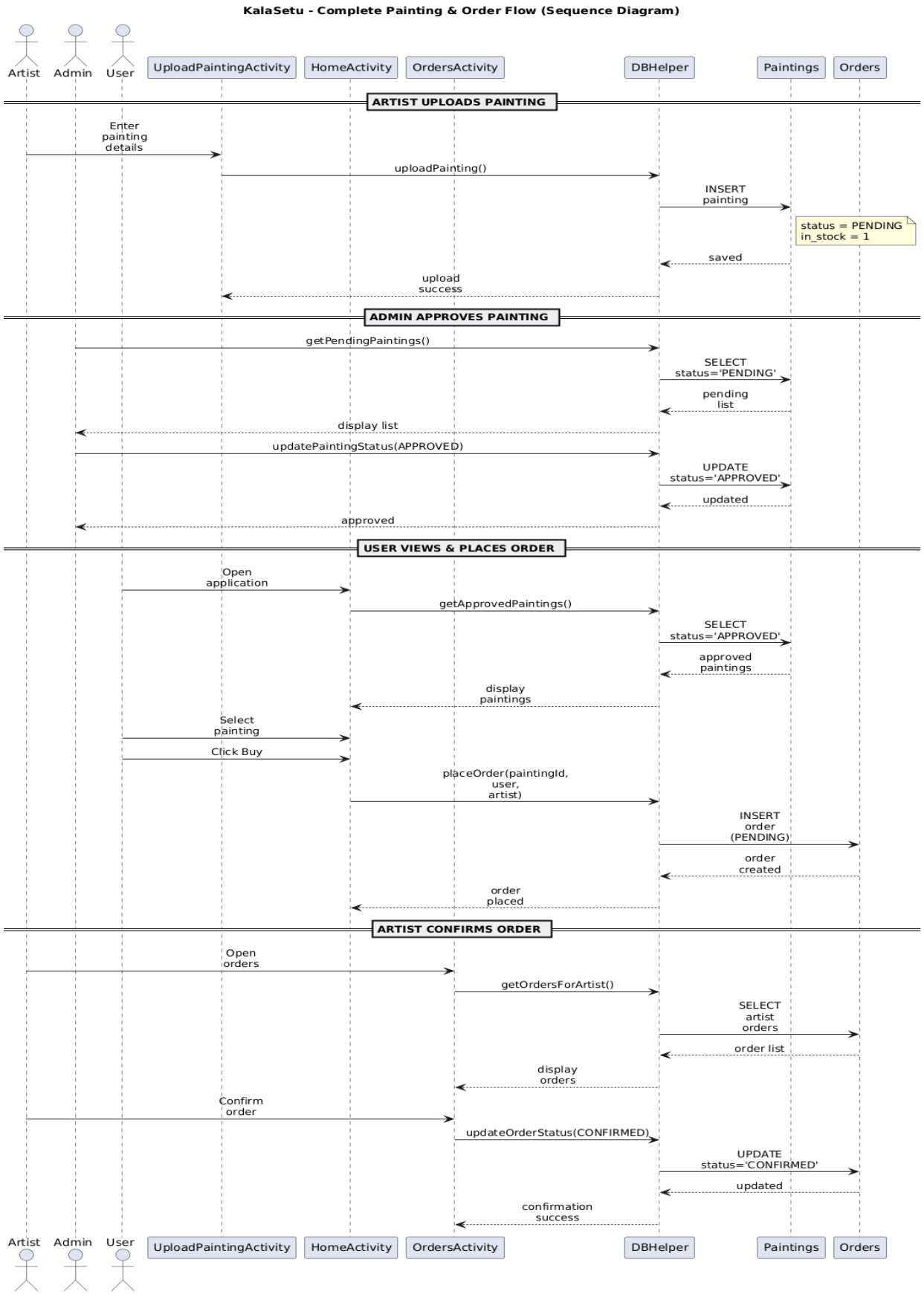
3. Class Diagram



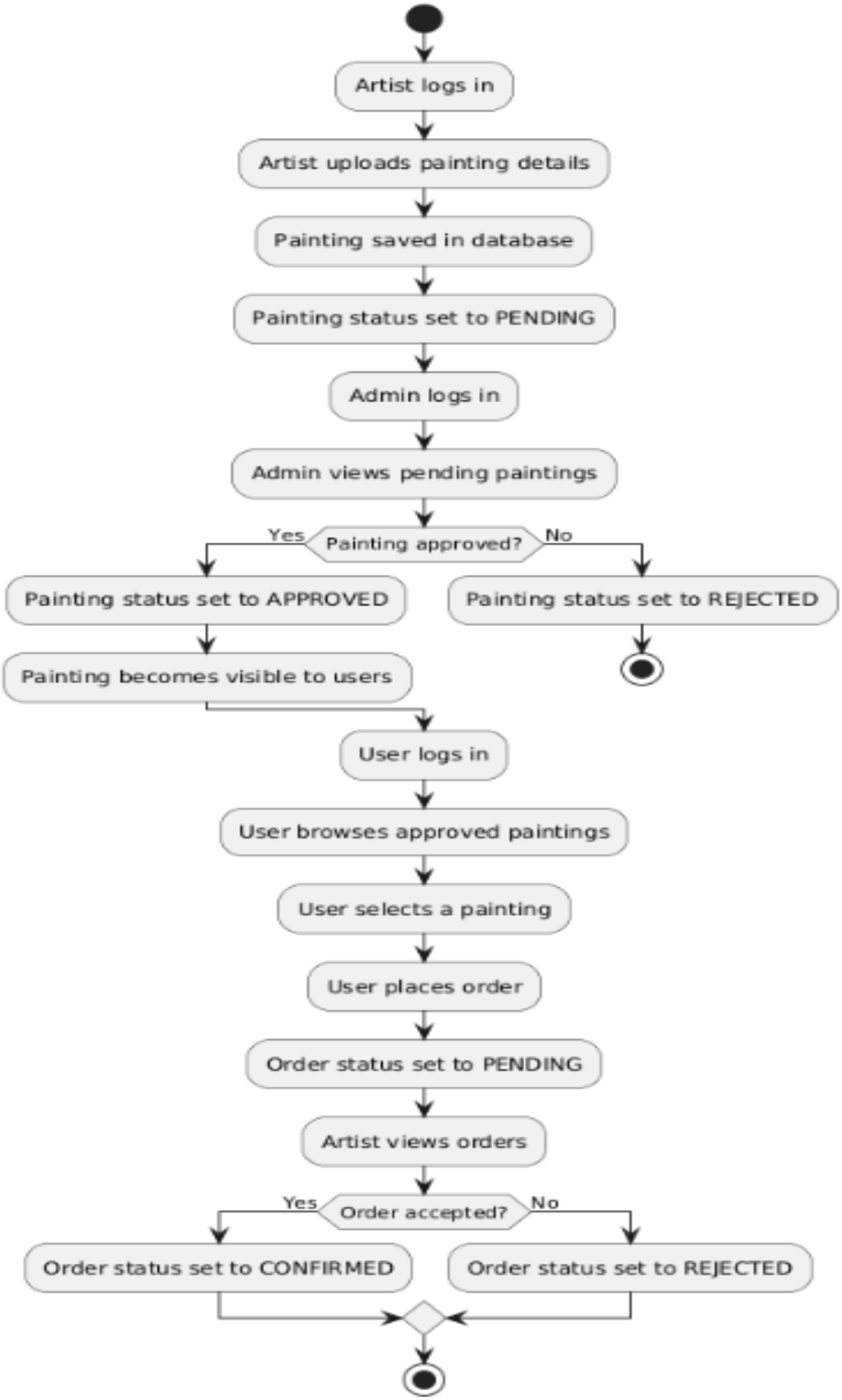
4.Object Diagram



5 Sequence Diagram



6. Activity Diagram



4.1System Coding

1. List of Modules

The Kalasetu application is implemented using a modular approach to ensure better organization, maintainability, and scalability. The system is divided into the following major modules:

1. User Module
2. Artist Module
3. Admin Module
4. Painting Management Module
5. Order Management Module
6. Database Management Module

Brief Description of Each Module

1. User Module
   - Handles registration, login, and authentication of users.
   - Allows users to view approved paintings.
   - Enables users to place orders and view order history.
2. Artist Module
   - Manages artist registration and login.
   - Allows artists to upload paintings.
   - Displays painting approval status.
   - Enables artists to view and manage received orders.
3. Admin Module
   - Provides administrative control over the application.
   - Allows admin to approve or reject paintings.
   - Manages users and artists.
   - Ensures quality and authenticity of artwork.
4. Painting Management Module
   - Handles painting upload, storage, and retrieval.
   - Maintains painting status such as Pending, Approved, or Rejected.
   - Controls visibility of paintings based on approval.
5. Order Management Module
   - Manages painting purchases by users.
   - Stores and tracks order details.
   - Allows order status updates by artists.
6. Database Management Module
   - Manages database creation and versioning.
   - Performs CRUD operations on all tables.
   - Ensures data consistency and integrity.

4.2 List Of Tables

1.User Table

Attribute	Data Type	Key	Description
user_id	INTEGER	PK	Unique user ID
name	TEXT	-	User name
username	TEXT	UNIQUE	Login username
password	TEXT	-	Login password
phone	TEXT	UNIQUE	User phone number
address	TEXT	-	User address
secret_code	TEXT	-	Password recovery code

2.Artist Table

Attribute	Data Type	Key	Description
artist_id	INTEGER	PK	Unique artist ID
name	TEXT	-	Artist name
username	TEXT	UNIQUE	Artist login username
password	TEXT	-	Login password
phone	TEXT	UNIQUE	Artist phone number
address	TEXT	-	Artist address
secret_code	TEXT	-	Password recovery code

3.Painting Table

Attribute	Data Type	Key	Description
painting_id	INTEGER	PK	Unique painting ID
artist_username	TEXT	FK	Uploaded artist
title	TEXT	-	Painting title
description	TEXT	-	Painting description
price	REAL	-	Painting price
image_uri	TEXT	-	Image path
status	TEXT	-	Approval status
in_stock	INTEGER	-	Availability
created_at	DATETIME	-	Upload time

4.Order Table

Attribute	Data Type	Key	Description
order_id	INTEGER	PK	Unique order ID
painting_id	INTEGER	FK	Painting reference
user_username	TEXT	FK	Buyer username
artist_username	TEXT	FK	Artist username
status	TEXT	-	Order status
created_at	DATETIME	-	Order date

4.3 Program description with code

The Kalasetu application is implemented using **Kotlin** and **SQLite database**. The core database operations such as table creation, data insertion, retrieval, update, and deletion are handled using a centralized database helper class named DBHelper. This class ensures structured data management and smooth interaction between different modules of the system.

1 Database Helper Class

The DBHelper class extends SQLiteOpenHelper and is responsible for database creation, version management, and execution of SQL queries.

Program Description

- Creates and manages the SQLite database named KalaSetu.db.
- Defines tables for users, artists, paintings, and orders.
- Provides methods for login, registration, painting upload, approval, and order management.
- Ensures safe database upgrade and downgrade handling.

Code:

```
class DBHelper(context: Context) :
   SQLiteOpenHelper(context, DATABASE_NAME, null, DATABASE_VERSION) {

   companion object {
      private const val DATABASE_NAME = "KalaSetu.db"
      private const val DATABASE_VERSION = 7
   }
```

2 Table Creation

This section creates all required tables when the database is initialized for the first time.

Program Description

- users table stores user details.
- artists table stores artist details.
- paintings table stores artwork uploaded by artists.
- orders table stores purchase information.

Code:

```
override fun onCreate(db: SQLiteDatabase) {

   db.execSQL("""
      CREATE TABLE users (
         user_id INTEGER PRIMARY KEY AUTOINCREMENT,
         name TEXT,
         username TEXT UNIQUE,
         password TEXT,
         phone TEXT UNIQUE,
         address TEXT,
         secret_code TEXT
      )
   """)
 db.execSQL("""
      CREATE TABLE artists (
```

```
            artist_id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT,
            username TEXT UNIQUE,
            password TEXT,
            phone TEXT UNIQUE,
            address TEXT,
            secret_code TEXT
        )
    """)
    db.execSQL("""
        CREATE TABLE paintings (
            painting_id INTEGER PRIMARY KEY AUTOINCREMENT,
            artist_username TEXT,
            title TEXT,
            description TEXT,
            price REAL,
            image_uri TEXT,
            status TEXT DEFAULT 'PENDING',
            in_stock INTEGER DEFAULT 1,
            created_at DATETIME DEFAULT CURRENT_TIMESTAMP
        )
    """)
    db.execSQL("""
        CREATE TABLE orders (
            order_id INTEGER PRIMARY KEY AUTOINCREMENT,
            painting_id INTEGER,
            user_username TEXT,
            artist_username TEXT,
            status TEXT DEFAULT 'PENDING',
            created_at DATETIME DEFAULT CURRENT_TIMESTAMP
        )
    """)
}
```

3 User Login and Authentication

Program Description

This method verifies user credentials during login by checking the username and password stored in the database.

Code:

```
fun checkUserLogin(username: String, password: String): Boolean =
    readableDatabase.rawQuery(
        "SELECT 1 FROM users WHERE username=? AND password=?",
        arrayOf(username, password)
    ).use { it.moveToFirst() }
```

4 Artist Upload Painting

Program Description

Artists can upload paintings by providing details such as title, description, price, and image. The painting status is set to **PENDING** until admin approval.

Code:

```
fun uploadPainting(
    artistUsername: String,
    title: String,
    description: String,
    price: Double,
    imageUri: String
): Boolean =
    writableDatabase.insert(
        "paintings",
        null,
        ContentValues().apply {
            put("artist_username", artistUsername)
            put("title", title)
            put("description", description)
            put("price", price)
            put("image_uri", imageUri)
            put("status", "PENDING")
            put("in_stock", 1)
        }
    ) != -1L
```

5 Admin Approval of Paintings

Program Description

The administrator reviews uploaded paintings and updates their status to either **APPROVED** or **REJECTED**.

Code:

```
fun updatePaintingStatus(id: Int, status: String): Boolean =
    writableDatabase.update(
        "paintings",
        ContentValues().apply { put("status", status) },
        "painting_id=?",
        arrayOf(id.toString())
    ) > 0
```

6 Order Placement

Program Description

When a user purchases a painting, a new order record is created in the database.

Code:

```
fun placeOrder(
    paintingId: Int,
    userUsername: String,
    artistUsername: String
): Boolean =
    writableDatabase.insert(
        "orders",
```

```
        null,
        ContentValues().apply {
            put("painting_id", paintingId)
            put("user_username", userUsername)
            put("artist_username", artistUsername)
            put("status", "PENDING")
        }
    ) != -1L
```

7 Order Status Update

Program Description

Artists can update the status of orders received for their paintings.

Code:

```
fun updateOrderStatus(orderId: Int, status: String): Boolean =
    writableDatabase.update(
        "orders",
        ContentValues().apply { put("status", status) },
        "order_id=?",
        arrayOf(orderId.toString())
    ) > 0
```

4.4 Test Case ,Test Data and Test Results

Test Case ID	Module	Why (Test Purpose)	Input (Test Data)	Expected Output	Observed Output	Remark
1	Admin Login	To check if only admin is allowed to login	Valid admin username and password	Admin access should be granted	Admin access was granted	PASS
2	Admin Login	To check admin login with invalid credentials	Invalid username or password	Access should be denied with error message	Access denied and error message displayed	PASS
3	Admin Login	To check admin login with blank input	Blank username and password	Access should be denied with validation message	Access denied and validation message shown	PASS
4	Artist Login	To check artist login with valid credentials	Valid artist username and password	Artist access should be granted	Artist access was granted	PASS
5	Artist Login	To check artist login with invalid credentials	Invalid artist credentials	Access should be denied with error message	Access denied and error message shown	PASS
6	Upload Painting	To check whether artist can upload painting	Valid title, description, price and image	Painting should be uploaded with PENDING status	Painting uploaded with PENDING status	PASS
7	Upload Painting	To check painting upload with missing details	Missing title or image	Painting should not be uploaded	Painting was not uploaded	PASS
8	Admin Approval	To check admin approval of painting	Pending painting selected by admin	Painting status should change to APPROVED	Painting approved successfully	PASS
9	View Paintings	To check user can view	Approved paintings available	Approved paintings should be	Approved paintings displayed	PASS

		approved paintings only		displayed		
10	Place Order	To check user can place order for painting	Valid painting selected by user	Order should be placed successfully	Order placed successfully	PASS
11	View Orders	To check artist can view received orders	Orders available for artist	Orders should be displayed	Orders displayed successfully	PASS
12	Update Order	To check artist can update order status	Valid order selected for update	Order status should be updated	Order status updated successfully	PASS

5. Future Enhancement

Although the Kalasetu application successfully fulfills its current objectives, several enhancements can be implemented in the future to improve functionality, scalability, and user experience.

1. Online Payment Gateway Integration

- A secure online payment system can be integrated to allow users to make direct digital payments using UPI, debit cards, credit cards, or net banking.

2. Delivery and Shipment Tracking

- A delivery tracking feature can be added to allow users to track the shipping status of purchased paintings in real time.

3. Rating and Review System

- Users can be allowed to rate paintings and provide feedback, helping other users make better purchasing decisions.

4. AI-Based Artwork Recommendation

- Artificial Intelligence can be used to recommend paintings based on user preferences, browsing history, and purchase behavior.

5. Multi-Platform Support

- The application can be extended to support web and iOS platforms, increasing accessibility and user reach.

6. Artist Dashboard and Analytics

- A dedicated dashboard can be provided for artists to view sales statistics, earnings, and performance insights.

7. Admin Analytics and Reports

- Advanced reporting tools can be added for admins to analyze user activity, sales trends, and system performance.

8. Cloud Database Integration

- Migrating from local SQLite to cloud-based databases such as Firebase or MySQL can improve scalability and data synchronization.

9. Notification System

- Push notifications can be introduced to notify users and artists about order updates, approvals, and new artwork uploads.

10. Enhanced Security Features

- Additional security measures such as password encryption, OTP-based authentication, and role-based access control can be implemented.

11. Wishlist and Favorites Feature

- Users can be allowed to save favorite paintings for future reference and easy purchase.

12.Advanced Search and Filter Options

- Search filters such as price range, artist name, category, and popularity can be added to improve usability.

13.Artwork Certification and Authenticity Tagging

- Digital certificates can be introduced to ensure the authenticity of paintings and build buyer confidence.

14.Multi-Language Support

- The application can support multiple languages to make it accessible to a wider and diverse user base.

15.Inventory and Stock Management Automation

- Automatic stock updates after each purchase can be implemented to avoid overselling of paintings.

Conclusion

The Kalasetu project has been successfully designed and implemented as a structured digital art marketplace that bridges the gap between artists and users through a secure, admin-controlled system. The application enables artists to upload paintings, administrators to verify and approve artwork, and users to browse and purchase only approved paintings. This approach ensures authenticity, quality control, and trust, making the platform reliable and user-friendly.

Challenges Faced and Overcome

During the development of the project, several challenges were encountered. One of the major challenges was designing a role-based system that clearly differentiates the functionalities of users, artists, and administrators. This was successfully addressed by implementing separate modules and controlled database access. Another challenge involved database design and data consistency, especially while managing relationships between users, artists, paintings, and orders. This challenge was overcome by creating a well-structured SQLite database with appropriate keys and constraints. Handling approval workflows and order tracking also required careful logic implementation, which was achieved through systematic coding and testing.

Learning Outcomes

The project provided valuable learning experiences in various technical and conceptual areas. It enhanced understanding of Android application development using Kotlin, SQLite database management, and modular software design. The project also strengthened knowledge of Software Engineering concepts such as requirement analysis, system design, implementation, testing, and documentation. Additionally, it improved problem-solving skills, debugging techniques, and the ability to convert real-world problems into effective software solutions.

Technical Success of the Project

From a technical perspective, the Kalasetu application operates successfully according to its defined objectives. All modules interact efficiently with the database, ensuring smooth data flow and reliable performance. The admin approval mechanism, secure authentication, painting management, and order processing functionalities work accurately as verified through comprehensive testing. The use of detailed test cases, test data, and test result tables confirms the functional correctness and stability of the system.

Overall Summary:

In conclusion, the Kalasetu project stands as a technically sound, well-tested, and practically relevant application. It successfully demonstrates the application of software engineering principles and mobile application development techniques. With the inclusion of proposed future enhancements, the system has strong potential to evolve into a scalable and intelligent digital art marketplace. The successful completion of this project reflects effective planning, technical competence, and a clear understanding of real-world application development.

----------xxxxxxxxxxxxxxxxxx----------